TALLINNA TEHNOLOOGIAKOLLEDŽ

Noorem tarkvaraarendaja

FUNKTSIOONIDE UURIMISE VEEBIRAKENDUS ASP.NET CORE MVC JA ENTITY FRAMEWORK CORE'I BAASIL

Lõputöö

Õpilane: Eesnimi Perenimi, õpperühm

Juhendaja: Eesnimi Perekonnanimi

Tallinn 2026

AUTORIDEKLARATSIOON

Deklareerin, et käesolev lõputöö, mis on minu iseseisva töö tulemus, on esitatud Tallinna Tehnoloogiakolledži lõputunnistuse taotlemiseks noorema tarkvaraarendaja õppekaval. Lõputöö alusel ei ole varem eriala lõputunnistust taotletud.

Autor Eesnimi Perenimi ……

(allkiri ja kuupäev)

Töö vastab kehtivatele nõuetele

Juhendaja Eesnimi Perenimi ……

(allkiri ja kuupäev)

SISUKORD

LÜHENDID JA MÕISTED.....

SISSEJUHATUS.....

RAKENDUSE KAVANDAMINE JA TEOREETILINE TAUST.....

Töö eesmärk, teemavaliku põhjendus ja olulisus.....

Funktsiooni uurimise matemaatiline alus.....

Lähteülesanne.....

Kasutatud tehnoloogiad ja vahendid.....

C# ja .NET 10.....

ASP.NET Core MVC.....

Entity Framework Core ja SQLite.....

MathNet.Symbolics.....

Chart.js, KaTeX ja MathLive.....

Arenduskeskkond ja tööriistad.....

Arendusmetoodika.....

Tegevus- ja ajakava.....

Rakenduse arhitektuur.....

Andmebaasi mudel.....

Kasutajaliidese kavandamine.....

LOODUD LAHENDUSE KIRJELDUS.....

Tehtud tööde maht.....

Kasutatud töövahendid ja paketid.....

Valitud meetodid ja töövõtted.....

Arendusprotsess.....

Mudeli ja andmebaasikonteksti loomine.....

Migratsioonid.....

CRUD-loogika.....
Funktsiooni analüüs.....
Lahenduskäigud.....
Graafik ja eelvaade.....
Versioonihaldus.....
Valminud lahenduse testimine.....
Kasutusjuhend.....
Paigaldamine ja käivitamine.....
Rakenduse kasutamine.....
KOKKUVÕTE.....
KASUTATUD ALLIKAD.....
LISAD.....
Lisa 1. Uurimisküsimused ja näidisvastused.....
Lisa 2. Graafiku rakendusliidese päringu ja vastuse näide.....

LÜHENDID JA MÕISTED

Lühend või mõiste	Selgitus
API	rakendusliides (*application programming interface*)
CDN	sisuedastusvõrk (*content delivery network*), kust brauser laadib väliseid teeke
CRUD	andmete loomine, lugemine, muutmine ja kustutamine (*create, read, update, delete*)
DI	sõltuvuste süstimine (*dependency injection*)
EF Core	Entity Framework Core, .NET-i objekt-relatsioonilise vastendamise raamistik
JSON	tekstipõhine andmevahetusvorming (*JavaScript Object Notation*)
LaTeX	matemaatiliste valemite märkimise keel, mida kasutab KaTeX
LINQ	C# keelde integreeritud päringud (*Language Integrated Query*)
LTS	pikaajalise toega väljalase (*long-term support*)
MVC	mudel-vaade-kontroller (*Model-View-Controller*) arhitektuurimuster
ORM	objekt-relatsiooniline vastendamine (*object-relational mapping*)
Code First	lähenemine, mille korral andmebaasi skeem tuletatakse koodis kirjeldatud mudeliklassidest
Migratsioon	EF Core'i loodud kood, mis muudab andmebaasi skeemi mudeli muudatuste järgi

SISSEJUHATUS

Funktsiooni uurimine on üks matemaatikaõppe kesksetest teemadest. Õppija peab etteantud valemi põhjal leidma funktsiooni määramis- ja muutumispiirkonna, nullkohad, positiivsus- ja negatiivsuspiirkonna, paarsuse, asümptoodid ning tuletise abil kasvamis- ja kahanemisvahemikud, ekstreemumkohad, kumerus- ja nõgususvahemikud ning käänupunktid. Ülesanne koosneb paljudest järjestikustest sammudest ja ühe sammu viga kandub edasi kõigisse järgmistesse vastustesse. Õpikutes ja kursuse ülesannetes on sageli antud üksnes lõppvastus, mistõttu iseseisval harjutamisel ei saa õppija teada, millises sammus ta eksis.

Olemasolevad graafikukalkulaatorid ja sümbolarvutuse veebiteenused joonistavad graafiku või arvutavad tuletise, kuid autori hinnangul ei esita need funktsiooni uurimist eesti koolis kasutatava ülesehituse ja tähistustega (X^+, X^-, $X\uparrow$, $X\downarrow$, kumer $\cap$ ja nõgus $\cup$) ega selgita lahenduskäiku eesti keeles. Sellest tulenes vajadus rakenduse järele, mis vastab kõigile funktsiooni uurimise küsimustele, näitab vastuste juures lihtsate sammudena lahenduskäiku ja võimaldab uuritud funktsioone hiljem uuesti vaadata.

Töö **eesmärk** on luua veebirakendus, mis sisestatud valemi põhjal uurib funktsiooni, joonistab selle graafiku koos oluliste punktidega, selgitab iga vastuse lahenduskäiku ning salvestab uuritud funktsioonid andmebaasi, kus neid saab vaadata, muuta ja kustutada.

Eesmärgi saavutamiseks seati järgmised ülesanded:

- koostada lähteülesanne ning valida sobivad tehnoloogiad ja töövahendid;
- kavandada rakenduse arhitektuur, andmemudel ja kasutajaliides;
- realiseerida funktsiooni analüüs ning vastuste lahenduskäikude koostamine;
- luua andmete haldamise (CRUD) funktsionaalsus, graafik ja valemi sisestamise vahendid;
- testida valminud rakendust ning koostada kasutusjuhend.

Rakendus loodi programmeerimiskeeles C# raamistiku ASP.NET Core MVC abil, andmed salvestatakse Entity Framework Core'i kaudu SQLite andmebaasi ning valemite sümbolarvutus tehakse teegiga MathNet.Symbolics. Töö koostamisel tugineti peamiselt kasutatud tehnoloogiate ametlikule dokumentatsioonile. Uurimisküsimuste valikul ja

tähistustes lähtuti matemaatikakursuse teemadest „Funktsiooni uurimine“, „Funktsiooni graafiku puutuja ja normaal“ ning „Funktsiooni uurimine tuletise abil“.

Töö koosneb kahest peatükist. Esimeses peatükis põhjendatakse teemavalikut, kirjeldatakse funktsiooni uurimise matemaatilist alust, lähteülesannet, kasutatud tehnoloogiaid, arendusmetoodikat ja ajakava ning rakenduse arhitektuuri, andmemudelit ja kasutajaliidese kavandamist. Teises peatükis kirjeldatakse loodud lahendust: tehtud tööde mahtu, töövahendeid, meetodeid ja arendusprotsessi etappe koos koodinäidetega, rakenduse testimist ning kasutusjuhendit. Töö lisades on esitatud uurimisküsimuste täielik loetelu koos näidisvastustega ja rakendusliidese päringu näide.

Töö koostamisel kasutati tehisaru tööriista Claude (Anthropic, 2026) rakenduse koodi osade kirjutamiseks ja silumiseks ning kirjaliku osa teksti mustandi koostamiseks. Tehisaru väljundit kontrolliti rakendust käivitades ja testides; kõik töös esitatud andmed, testitulemused ja kuvatõmmised pärinevad töötavast rakendusest ning teksti kontrollis, täiendas ja kohandas autor.

1. RAKENDUSE KAVANDAMINE JA TEOREETILINE TAUST

Peatükis kirjeldatakse töö eesmärki ja teemavaliku põhjendust, funktsiooni uurimise matemaatilist alust, rakenduse lähteülesannet, kasutatud tehnoloogiaid, arendusmetoodikat ja ajakava ning rakenduse arhitektuuri, andmemudeli ja kasutajaliidese kavandamist.

1.1. Töö eesmärk, teemavaliku põhjendus ja olulisus

Teema valiti seetõttu, et see ühendab tarkvaraarenduse ja matemaatika: rakendus peab mitte ainult andmeid salvestama ja kuvama, vaid ka valemit mõistma, seda analüüsima ja tulemuse õppijale arusaadaval kujul esitama. Selline ülesanne nõuab nii veebirakenduse arhitektuuri ja andmebaasi kavandamise oskust kui ka algoritmilist mõtlemist.

Rakenduse olulisus seisneb selles, et õppija saab oma käsitsi leitud vastust kohe võrrelda ja vea korral vaadata lahenduskäiku. Erinevalt tavalisest graafikukalkulaatorist ei piirdu rakendus vastusega: iga vastuse juures on nupp „Lahendus“, mis näitab lahendust samm-sammult, võimaluse korral mitmel viisil (näiteks ruutvõrrandi lahendamine diskriminandi, taandatud ruutvõrrandi valemi ja Viète’i teoreemi abil). Õpetaja saab rakendust kasutada näidete koostamiseks ja ülesannete vastuste kontrollimiseks.

Rakenduse sihtrühm on eelkõige gümnaasiumi ja kutseõppe õppijad, kes õpivad funktsiooni uurimist ja tuletist, ning nende õpetajad. Sellest tulenevalt peab rakendus olema eestikeelne, kasutama koolis tavapäraseid tähistusi ja esitama arvud võimaluse korral täpsel kujul (näiteks $-\sqrt{3}$ ja $\pi/2$ ligikaudsete kümnendmurdude asemel).

1.2. Funktsiooni uurimise matemaatiline alus

Funktsiooni uurimisel leitakse funktsiooni $f(x)$ omadused, mis koos kirjeldavad graafiku kuju. Rakendus vastab kokku 19 küsimusele, mis jagunevad kahte rühma: funktsiooni uurimine ilma tuletiseta ja uurimine tuletise abil (vt lisa 1). Tabelis 1 on esitatud peamised omadused ja nende leidmise põhimõte.

Tabel 1. Funktsiooni uuritavad omadused ja nende leidmise põhimõte

Omadus	**Tähis**	**Leidmise põhimõte**
Määramispiirkond	X	argumendi väärtused, mille korral $f(x)$ on määratud (nimetaja $\neq 0$, juuritav ≥ 0, logaritmitav > 0)

Omadus	Tähis	Leidmise põhimõte
Nullkohad	X_0	võrrandi $f(x) = 0$ lahendid
Positiivsus- ja negatiivsuspiirkond	X^+, X^-	intervallmeetod: nullkohad jagavad määramispiirkonna vahemikeks, märk leitakse proovipunktiga
Paarsus	–	$f(-x)$ võrdlemine $f(x)$-ga; eeldab nullpunkti suhtes sümmeetrilist määramispiirkonda
Asümptoodid	–	piirväärtused poolustel (vertikaalne) ja lõpmatuses (horisontaalne, kaldasümptoot)
Kasvamis- ja kahanemisvahemikud	$X\uparrow$, $X\downarrow$	tuletise $f'(x)$ märk vahemikes, mille piirid on kriitilised punktid
Ekstreemumkohad	X_e	kriitilised punktid, kus f' märk muutub ($+ \to -$ maksimum, $- \to +$ miinimum)
Kumerus ja nõgusus	$\cap$, $\cup$	teise tuletise $f''(x)$ märk ($\cap$, kui $f'' < 0$; $\cup$, kui $f'' > 0$)
Käänupunktid	–	$f''(x) = 0$ lahendid, kus f'' märk muutub

Kooliülesannetes leitakse vastused analüütiliselt: lahendatakse võrrandeid, tegurdatakse avaldisi ja arvutatakse piirväärtusi. Arvuti jaoks on analüütiline lahendamine keeruline, sest iga võrranditüübi jaoks on vaja eraldi võtet ning paljudel võrranditel (näiteks $x^3 + x + 1 = 0$) koolivõtetega lahendit leida ei saa. Seetõttu kasutati rakenduses kahte teineteist täiendavat lähenemist.

Esimene on ***numbriline analüüs***. Funktsiooni ja selle tuletiste väärtused arvutatakse tihedas punktivõrgus ning nullkohad leitakse märgimuutuse järgi: kui kahes naaberpunktis on funktsiooni väärtused erineva märgiga, asub nende vahel nullkoht, mis täpsustatakse poolitusmeetodiga. Poolitusmeetodi igal sammul jagatakse vahemik pooleks ja jäetakse alles see pool, mille otstes on funktsiooni märk erinev, kuni vahemik on piisavalt väike. Nullkohad, kus funktsioon märki ei muuda (näiteks x^2 puutepunkt kohal 0), leitakse kuldlõike otsinguga funktsiooni absoluutväärtuse miinimumina. Märgipiirkonnad leitakse intervallmeetodiga ning asümptoodid piirväärtuste abil.

Teine on ***sümbolarvutus***. Valem teisendatakse avaldise puuks, millest leitakse tuletised sümbolselt (näiteks $x^3 - 3x \to 3x^2 - 3 \to 6x$) ning mille põhjal koostatakse lahenduskäik koolivõtetega. Sümbolarvutuse tulemusi kontrollitakse alati numbrilise analüüsi vastu: kui täpne lahend ei lange kokku numbriliselt leitud väärtusega, lahenduskäiku ei näidata. Kui ükski koolivõte võrrandile ei sobi, näidatakse numbrilist lahendamist. Selline kontroll tagab, et rakendus ei esita kunagi vale täpset vastust.

1.3. Lähteülesanne

Lähteülesanne koostati enne arendamise algust ning seda täiendati arenduse käigus, kui lisandusid uued uurimisküsimused (vt alapeatükk 1.6). Rakenduse funktsionaalsed nõuded on esitatud tabelis 2.

Tabel 2. Rakenduse funktsionaalsed nõuded

Nr	Nõue
F1	Kasutaja saab sisestada funktsiooni valemi tekstina (nt `x^3 - 3*x`) või visuaalses valemiväljas valemi klaviatuuri abil.
F2	Rakendus kontrollib valemit ja annab vea korral eestikeelse veateate.
F3	Rakendus vastab funktsiooni uurimise 19 küsimusele ning määrab funktsiooni liigi.
F4	Rakendus joonistab graafiku, millel on märgitud nullkohad, ekstreemumid, käänupunktid, lõikepunkt *y*-teljega ja asümptoodid; graafikut saab suumida ja liigutada.
F5	Loomise ja muutmise vormis uuenevad graafik ja vastused valemi sisestamise ajal (eelvaade).
F6	Iga vastuse juures saab avada lahenduskäigu; võimaluse korral näidatakse mitut lahendusviisi.
F7	Uuritud funktsiooni saab salvestada, vaadata, muuta ja kustutada (CRUD).
F8	Salvestatud funktsioonid on nimekirjas rühmitatud liikide kaupa ning neid saab liigi järgi filtreerida.
F9	Rakenduses on teooria leht kursuse teemade valemitega, kust saab näite avada rakenduses.

Lisaks funktsionaalsetele nõuetele seati rakendusele tabelis 3 esitatud mittefunktsionaalsed nõuded.

Tabel 3. Rakenduse mittefunktsionaalsed nõuded

Nr	Valdkond	Nõue
M1	Keel	kasutajaliides, veateated ja lahenduskäigud on eesti keeles ning kasutavad koolis tavapäraseid tähistusi
M2	Jõudlus	ühe funktsiooni analüüs koos lahenduskäikudega võtab alla ühe sekundi, et eelvaade uueneks sisestamise ajal
M3	Õigsus	täpne vastus kontrollitakse numbrilise vastu; kontrolli mittetäitev vastus jäetakse näitamata
M4	Paigaldamine	rakendus käivitub ühe käsuga ilma eraldi andmebaasiserverita; andmebaas luuakse automaatselt
M5	Töökindlus	kui väline teek (nt MathLive) ei laadi, jääb alles tavaline tekstiväli ja rakendus töötab edasi
M6	Turvalisus	vormid on kaitstud võltsitud päringute (CSRF) eest ja sisend valideeritakse serveris
M7	Hooldatavus	kood on jagatud kihtideks; klasside ja muutujate nimed on eestikeelsed ja kooskõlas valdkonna mõistetega

1.4. Kasutatud tehnoloogiad ja vahendid

Tehnoloogiate valikul lähtuti lähteülesandest ning õppekavas omandatud oskustest. Järgmistes alapeatükkides kirjeldatakse valitud tehnoloogiaid ja põhjendatakse nende valikut; kasutatud pakettide ja teekide versioonid on esitatud alapeatükis 2.2.

1.4.1. C# ja .NET 10

Rakendus on kirjutatud programmeerimiskeeles C#, mis on staatiliselt tüübitud objektorienteeritud keel ja .NET-platvormi peamine keel. C# valiti, kuna see on õppekava põhikeel ning sellel on hea tugi nii veebiarenduseks kui ka andmebaasidega töötamiseks. Sihtplatvormiks valiti .NET 10, mis on pikaajalise toega (LTS) väljalase ja mida toetatakse kolm aastat (Microsoft, 2025). Pikaajaline tugi tähendab, et rakendust ei pea lähiaastatel uuemale platvormile üle viima. Projektis kasutatakse ka keele uuemaid võimalusi, nagu kirjed (*record*), mustrisobitus ja kogumiavaldised, mis muudavad andmeklasside ja analüüsikoodi lühemaks.

1.4.2. ASP.NET Core MVC

Veebirakenduse raamistikuks valiti ASP.NET Core MVC. MVC-muster jagab rakenduse kolmeks osaks: mudel kirjeldab andmeid ja äriloogikat, vaade kuvab kasutajaliidest ning kontroller töötleb kasutaja päringuid, suhtleb mudeliga ja valib vaate (Microsoft, s.a.-c). Selline jaotus muudab koodi hõlpsamini testitavaks ja hooldatavaks, sest iga osa vastutab ühe asja eest.

ASP.NET Core'i oluline osa on sisseehitatud sõltuvuste süstimine: teenused registreeritakse rakenduse käivitamisel ning raamistik annab need kontrolleritele konstruktori kaudu (Microsoft, s.a.-a). Rakenduses registreeritakse nii andmebaasikontekst kui ka funktsiooni analüüsija. Lisaks pakub raamistik kaitset võltsitud päringute (CSRF) vastu: vormidesse lisatakse peidetud märgis, mida server POST-päringu korral kontrollib (Microsoft, s.a.-e). MVC kõrval kasutatakse samas rakenduses ka JSON-vastuseid tagastavat API-kontrollerit, mida brauseris töötav JavaScript kutsub eelvaate uuendamiseks.

1.4.3. Entity Framework Core ja SQLite

Andmebaasiga suhtlemiseks kasutatakse Entity Framework Core'i, mis on .NET-i objekt-relatsioonilise vastendamise (ORM) raamistik. EF Core võimaldab töötada

andmebaasiga .NET-i objektide kaudu ning kirjutada päringuid LINQ-i abil, mistõttu enamikku andmetele juurdepääsu koodist ei pea SQL-is kirjutama (Microsoft, s.a.-d). Rakenduses kasutatakse Code First lähenemist: andmebaasi skeem tuletatakse mudeliklassist ning skeemi muudatused tehakse migratsioonidega, mis hoiavad andmebaasi skeemi mudeliga kooskõlas ilma olemasolevaid andmeid kaotamata (Microsoft, s.a.-b).

Andmebaasimootoriks valiti SQLite. SQLite on protsessisisene teek, mis realiseerib iseseisva, serverita, seadistamist mittevajava ja tehingupõhise SQL-andmebaasimootori (SQLite, s.a.). Kuna rakenduse andmemaht on väike ja andmebaasi kasutab üks rakendus, ei ole eraldi andmebaasiserverit vaja. Andmebaas on üks fail (`funktsioonid.db`), mis luuakse rakenduse esmakäivitusel automaatselt; see täidab mittefunktsionaalse nõude M4. Kuna EF Core toetab mitut andmebaasimootorit, saab vajaduse korral hiljem üle minna näiteks SQL Serverile, muutes peamiselt andmebaasipakkujat ja ühendusteksti.

1.4.4. MathNet.Symbolics

Valemite töötlemiseks kasutatakse teeki Math.NET Symbolics, mis on F#-is kirjutatud avatud lähtekoodiga sümbolarvutuse teek .NET-i jaoks ja mida levitatakse MIT/X11 litsentsi alusel (Math.NET, s.a.). Teek teisendab tekstina antud valemi avaldise puuks, leiab tuletise sümbolselt, lihtsustab avaldisi ja kompileerib avaldise .NET-i funktsiooniks, mida saab kiiresti paljudes punktides arvutada. Kõiki neid võimalusi kasutatakse funktsiooni analüüsis. Avaldise puud kasutatakse ka funktsiooni liigi määramiseks ja koolipärase LaTeX-kuju koostamiseks.

1.4.5. Chart.js, KaTeX ja MathLive

Kasutajaliidese matemaatilised osad on realiseeritud kolme JavaScripti teegiga, mis laaditakse CDN-ist. Graafik joonistatakse teegiga Chart.js, mis kasutab HTML-i lõuendielementi ja toetab muu hulgas joon- ja hajuvusdiagramme ning nende kombineerimist (Chart.js, s.a.). Graafiku suumimiseks ja liigutamiseks hiire, kerimisratta ja puuteekraaniga kasutatakse lisandmoodulit chartjs-plugin-zoom (Chart.js, s.a.-b).

Valemid kuvatakse teegiga KaTeX, mis joonistab LaTeX-kujul antud valemid brauseris ilma serveripoolse töötluseta (KaTeX, s.a.). Lahenduskäigud koostatakse serveris LaTeX-kujul ning KaTeX kuvab need murrujoonte, juuremärkide ja astmetega nagu vihikus. Valemi sisestamiseks kasutatakse teeki MathLive, mis pakub veebikomponenti matemaatiliste valemite redigeerimiseks (MathLive, s.a.). Visuaalses valemiväljas näevad aste ja murd välja

nagu vihikus ning valem teisendatakse rakenduse JavaScripti abil tekstikujule, mida server mõistab.

1.4.6. Arenduskeskkond ja tööriistad

Arendamiseks kasutati Visual Studiot ja Visual Studio Code'i. Andmebaasi migratsioonide loomiseks kasutati käsureatööriista `dotnet-ef`, mis on lisatud projekti kohalikku tööriistamanifesti. Tänu sellele saab iga arendaja tööriista sama versiooni paigaldada käsuga `dotnet tool restore`. NuGeti paketid hallatakse projektifailis ning rakendus käivitatakse käsuga `dotnet run`.

1.5. Arendusmetoodika

Rakendus arendati üksi, seetõttu ei kasutatud meeskonnale mõeldud raamistikku. Näiteks Scrum eeldab rolle (tooteomanik, Scrum Master, arendajad), kindla pikkusega sprinte ning sündmusi, nagu sprindi planeerimine ja tagasivaade (Schwaber & Sutherland, 2020). Ühe arendaja projektis oleks selline korraldus liiga kulukas.

Töö tehti iteratiivselt ja inkrementaalselt, kasutades Kanbani põhimõtteid: ülesanded olid ühes tööjärjekorras, korraga tehti üht ülesannet ning iga iteratsiooni lõpus oli töötav rakendus, millele lisati uus võimalus. Esimeses iteratsioonis loodi CRUD-funktsionaalsus ja Teema 8 küsimuste analüüs, järgmistes lisati uurimine tuletise abil, funktsiooni liigid, lahenduskäigud, valemi klaviatuur, teooria leht ja lahenduskäikude joonised. Iga uue võimaluse järel kontrolliti rakendust kursuse ülesannete funktsioonidega (vt alapeatükk 2.5). Andmemudeli muudatused tehti migratsioonidena, mistõttu on andmebaasi arengut võimalik migratsioonide järgi jälgida (tabel 9).

1.6. Tegevus- ja ajakava

Praktilise töö ajakava on esitatud tabelis 4. Etappide kuupäevad põhinevad migratsioonide ajatemplitel ja failide muutmise aegadel.

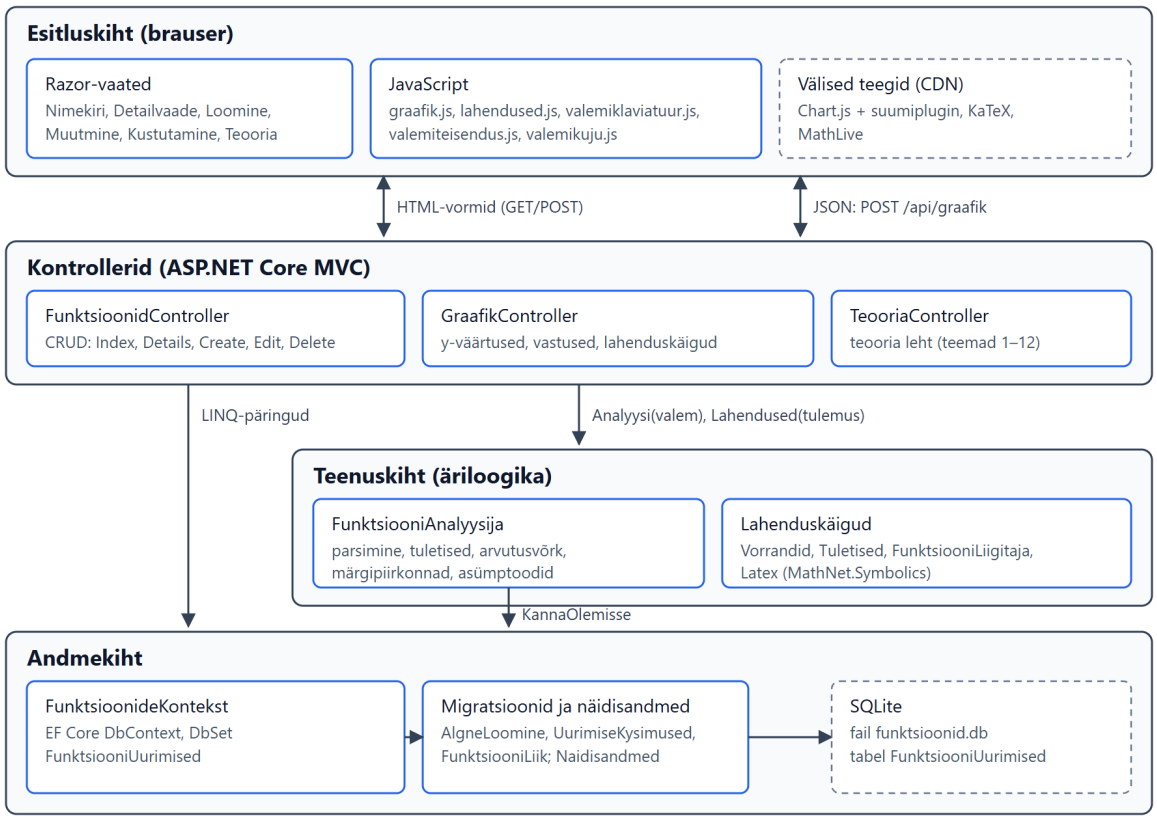
Tabel 4. Tegevus- ja ajakava

Etapp	Aeg	Tulemus
1. Analüüs ja kavandamine	kuni 30.09.2026	lähteülesanne, tehnoloogiate valik, arhitektuuri ja andmemudeli kavand
2. Projekti loomine ja CRUD	30.09.2026	ASP.NET Core MVC projekt, mudel, andmebaasikontekst, migratsioon AlgneLoomine, CRUD-vaated

Etapp	Aeg	Tulemus
3. Funktsiooni analüüs ja graafik	30.09–07.10.2026	Teema 8 küsimused, graafik, eelvaade
4. Uurimine tuletise abil ja liigid	07.10.2026	Teema 11 küsimused, migratsioonid UurimiseKysimused ja FunktsiooniLiik, nimekirja rühmitamine
5. Lahenduskäigud ja kasutajaliides	07.10–12.10.2026	võrrandite ja tuletiste lahenduskäigud, märgitabelid, valemi klaviatuur, teooria leht
6. Joonised ja testimine	12.10–14.10.2026	lahenduskäikude joonised, funktsionaalne testimine, vigade parandamine
7. Kirjalik osa ja kaitsmine	14.10.2026 – kaitsmine	lõputöö kirjalik osa, ettekanne

1.7. Rakenduse arhitektuur

Rakendus on üles ehitatud kihtidena (joonis 1). Iga kiht kasutab ainult endast allpool olevat kihti, mistõttu saab ühe kihi sisu muuta teisi kihte puudutamata.



Joonis 1. Rakenduse arhitektuur

Esitluskiht koosneb Razor-vaadetest ja brauseris töötavast JavaScriptist. Vaated luuakse serveris ning brauserisse saadetakse valmis HTML. JavaScript lisab lehele interaktiivsuse:

graafiku joonistamise, eelvaate uuendamise valemi sisestamise ajal, lahenduskäikude avamise ja visuaalse valemivälja.

Kontrollerid võtavad vastu HTTP-päringud. `FunktsioonidController` realiseerib CRUD-toimingud ja tagastab vaated, `GraafikController` tagastab graafiku andmed, vastused ja lahenduskäigud JSON-kujul ning `TeooriaController` kuvab teooria lehe. Kontrollerites äriloogikat ei ole: need kutsuvad teenuseid ja andmekihti ning edastavad tulemuse vaatele.

Teenuskiht sisaldab rakenduse äriloogikat. Klass `FunktsiooniAnalyysija` parsib valemi ja leiab kõik vastused; osaklassid ja abiklassid koostavad lahenduskäigud (`Vorrandid`, `Tuletised`), määravad funktsiooni liigi (`FunktsiooniLiigitaja`) ja vormindavad avaldised LaTeX-kujul (`Latex`). Analüüsija ei hoia olekut, seetõttu registreeritakse see sõltuvuste süstimises ühe eksemplarina (*singleton*), mida kõik päringud jagavad.

Andmekiht koosneb andmebaasikontekstist `FunktsioonideKontekst`, migratsioonidest ja näidisandmetest. Andmebaasikontekst registreeritakse päringupõhise elueaga, nii et iga HTTP-päring saab oma konteksti.

1.8. Andmebaasi mudel

Andmemudel loodi Code First lähenemisega. Andmebaasis on üks tabel `FunktsiooniUurimised`, mis vastab mudeliklassile `FunktsiooniUurimine`. Iga kirje sisaldab valemit, funktsiooni liiki, kõigi uurimisküsimuste vastuseid tekstina ning loomise aega (tabel 5).

Tabel 5. Tabeli FunktsiooniUurimised väljad

Väli	Tüüp	Selgitus
Id	INTEGER, primaarvõti	kirje identifikaator (automaatselt suurenev)
Valem	TEXT(200), kohustuslik	valem tekstikujul, nt `x^3 - 3*x`
Liik	TEXT(30), indeks	funktsiooni liigi võti, nt `kuup`
Maaramispiirkond … Asumptoodid	TEXT(500)	Teema 8 küsimuste vastused (9 välja)
Tuletis … Kaanupunktid	TEXT(500)	Teema 11 küsimuste vastused (9 välja)
LuodudAeg	TEXT (kuupäev), indeks	loomise aeg UTC-s; nimekirja järjestamiseks

Mudelis ei ole teisi olemeid ega võõrvõtmeid. Kõik vastused tulenevad üheselt valemist ning analüüsitakse kirje salvestamisel, seetõttu ei ole neid vaja eraldi tabelitesse jagada. Vastuste

salvestamine (mitte iga kord uuesti arvutamine) võimaldab nimekirjas kuvada tuletist ja ekstreemumeid ilma iga rea korral analüüsi uuesti tegemata. Lahenduskäike andmebaasi ei salvestata: need on mahukad, arvutatakse kiiresti ning muutuvad, kui lahenduskäikude koostamist täiendatakse. Funktsiooni liikide loetelu on koodis staatilise loendina (`FunktsiooniLiigid`), sest liigid on rakenduse loogika osa ja neid kasutaja ei muuda.

Tabeli nimi ja indeksid on määratud andmebaasikonteksti meetodis `OnModelCreating` (joonis 2). Indeks väljal `Liik` kiirendab nimekirja filtreerimist liigi järgi ning indeks väljal `LuodudAeg` järjestamist loomise aja järgi.

```
public class FunktsioonideKontekst(DbContextOptions<FunktsioonideKontekst>
valikud) : DbContext(valikud)
{
    public DbSet<FunktsiooniUurimine> FunktsiooniUurimised =>
Set<FunktsiooniUurimine>();

    protected override void OnModelCreating(ModelBuilder mudel)
    {
        mudel.Entity<FunktsiooniUurimine>(olem =>
        {
            olem.ToTable("FunktsiooniUurimised");
            olem.HasIndex(f => f.LuodudAeg);
            olem.HasIndex(f => f.Liik);
        });
    }
}
```

Joonis 2. Andmebaasikontekst ja mudeli seadistamine (Andmed/FunktsioonideKontekst.cs)

Väljade pikkus ja kohustuslikkus on määratud mudeliklassis andmemärgenditega (`[Required]`, `[MaxLength]`), mida EF Core kasutab skeemi loomisel. Kasutaja sisestatud andmete jaoks on eraldi vormimudel `FunktsiooniVorm`, milles on ainult valem ja selle valideerimisreeglid; kõik ülejäänud väljad arvutab server. Nii ei saa kasutaja vormi kaudu vastuseid ise määrata.

1.9. Kasutajaliidese kavandamine

Kasutajaliides kavandati võimalikult lihtsaks: päises on rakenduse nimi, lüliti „Ilus valem“ ning lingid nimekirjale, uue funktsiooni loomisele ja teooria lehele. Rakenduse lehed on esitatud tabelis 6.

Tabel 6. Rakenduse lehed

Aadress	**Leht**	**Sisu**
/	Nimekiri	salvestatud funktsioonid liikide kaupa, liigi filtrid, lingid muutmisele ja kustutamisele
/Funktsioonid/Details/{id}	Detailvaade	valem, liik, graafik, vastused kahes kaardis ja lahenduskäigud
/Funktsioonid/Create	Uus funktsioon	valemiväli koos klaviatuuriga, eelvaate graafik ja vastused
/Funktsioonid/Edit/{id}	Muutmine	nagu loomise vorm, eeltäidetud valemiga
/Funktsioonid/Delete/{id}	Kustutamine	kustutamise kinnitus
/Teooria	Teooria	kursuse teemad 1–12 valemitega, nupud „Proovi“

Detailvaade ning loomise ja muutmise vorm on koostatud samadest osavaadetest: graafik (`_Graafik`), vastused (`_Vastused`) ja valemiväli (`_ValemiVali`). Detailvaates näidatakse salvestatud vastuseid, eelvaates täidab samad väljad JavaScript serveri vastuse põhjal. Vastused on jagatud kahte kaarti vastavalt küsimuste rühmale ning iga vastuse juures on nupp „Lahendus“, mis avab lahenduskäigu vastuse all. Mitme lahendusviisi korral kuvatakse viisid vahelehtedena.

Graafiku kohal on väljad *x*-vahemiku ja sammu muutmiseks ning nupud suumimiseks ja algvaatele naasmiseks („Sobita“). Olulised punktid on graafikul eri kujuga märkidega: nullkohad õõnsa ringiga, maksimumid kolmnurgaga üles, miinimumid kolmnurgaga alla ja käänupunktid rombiga, ning nende juurde on kirjutatud koordinaadid.

2. LOODUD LAHENDUSE KIRJELDUS

Peatükis kirjeldatakse tehtud tööde mahtu, kasutatud töövahendeid ja meetodeid, arendusprotsessi etappe koos koodinäidetega, rakenduse testimist ning kasutusjuhendit.

2.1. Tehtud tööde maht

Rakenduse lähtekoodi maht on esitatud tabelis 7. Suurima osa koodist moodustab teenuskiht, eriti funktsiooni analüüs ja lahenduskäikude koostamine. Migratsioonide automaatselt genereeritud kood ei ole mahu hulka arvestatud.

Tabel 7. Lähtekoodi maht

Osa	Failid	Ridu
C# (teenused, kontrollerid, mudelid, andmekiht, käivitus)	16	4 244
JavaScript (graafik, lahenduskäigud, valemiväli, teisendused)	5	1 224
Razor-vaated	13	826
CSS	1	364
Kokku	35	6 658

Suurimad üksikud failid on `FunktsiooniAnalyysija.cs` (funktsiooni numbriline analüüs, umbes 1 100 rida), `FunktsiooniAnalyysija.Lahendused.cs` (lahenduskäigud kõigile küsimustele, umbes 1 050 rida) ja `Vorrandid.cs` (võrrandite lahendamine koolivõtetega, umbes 700 rida). Kõige rohkem aega kulus lahenduskäikude koostamisele, sest iga võrranditüübi jaoks tuli kirjutada eraldi lahendusvõte ja kontrollida selle tulemust.

2.2. Kasutatud töövahendid ja paketid

Projektis kasutatud NuGeti paketid ja muud sõltuvused on esitatud tabelis 8. JavaScripti teegid laaditakse CDN-ist kindla versiooninumbriga, et teegi uus versioon ei muudaks rakenduse käitumist ootamatult.

Tabel 8. Kasutatud paketid ja teegid

Pakett või teek	Versioon	Kasutus
MathNet.Symbolics	0.24.0	NuGet; valemite sümbolarvutus
Microsoft.EntityFrameworkCore.Sqlite	10.0.12	NuGet; EF Core’i SQLite pakkuja
Microsoft.EntityFrameworkCore.Design	10.0.12	NuGet; migratsioonide loomine arenduse ajal

Pakett või teek	Versioon	Kasutus
dotnet-ef	10.0.12	kohalik .NET-i tööriist; migratsioonide käsud
Chart.js	4.4.4	CDN; graafik
chartjs-plugin-zoom, Hammer.js	2.2.0; 2.0.8	CDN; suumimine ja puutežestid
KaTeX	0.16.11	CDN; valemite kuvamine
MathLive	0.111.0	CDN; visuaalne valemiväli

2.3. Valitud meetodid ja töövõtted

Arendamisel kasutati järgmisi meetodeid ja töövõtteid:

- ***Code First ja migratsioonid*** – andmebaasi skeem tuletatakse mudeliklassist ning iga muudatus tehakse migratsioonina, mis rakendatakse käivitamisel automaatselt;
- ***kihiline arhitektuur ja sõltuvuste süstimine*** – kontrollerid saavad andmebaasikonteksti ja analüüsija konstruktori kaudu;
- ***kahekordne valideerimine*** – vormimudeli andmemärgendid kontrollivad lubatud märke, analüüsija kontrollib valemi sisu ja annab täpse eestikeelse veateate;
- ***sümbolarvutuse ja numbrilise analüüsi ühendamine*** – täpne tulemus kontrollitakse alati numbrilise vastu;
- ***järkjärguline täiustamine*** – rakenduse põhifunktsioonid töötavad ka siis, kui mõni väline JavaScripti teek ei laadi;
- ***eestikeelne nimetamine*** – klasside, meetodite ja muutujate nimed on eesti keeles ja kasutavad valdkonna mõisteid (nt `Nullkohad`, `Kaanupunktid`, `MargiVahemikud`), mis teeb koodi ja matemaatika vastavuse jälgitavaks.

2.4. Arendusprotsess

Alapeatükis kirjeldatakse arenduse peamisi etappe ja nende lahendusi koos koodinäidetega.

2.4.1. Mudeli ja andmebaasikonteksti loomine

Esimese sammuna loodi ASP.NET Core MVC projekt ja lisati EF Core'i paketid. Seejärel loodi mudeliklass `FunktsiooniUurimine` ja andmebaasikontekst `FunktsioonideKontekst` (joonis 2). Ühendustekst on konfiguratsioonifailis `appsettings.json` (`Data`

`Source=funktsioonid.db`) ning kontekst registreeritakse failis `Program.cs` meetodiga `AddDbContext`. Analüüsija registreeritakse samas failis meetodiga `AddSingleton`.

2.4.2. Migratsioonid

Andmebaasi skeem loodi ja seda muudeti migratsioonidega. Migratsioonide käsud käivitati käsurealt projekti kaustas (joonis 3).

```
dotnet tool restore
dotnet ef migrations add AlgneLoomine
dotnet ef migrations add UurimiseKysimused
dotnet ef migrations add FunktsiooniLiik
dotnet ef database update
```

Joonis 3. Migratsioonide loomise ja rakendamise käsud

Projekti arenduse käigus loodi kolm migratsiooni (tabel 9).

Tabel 9. Projekti migratsioonid

Migratsioon	Kuupäev	Muudatus
AlgneLoomine	30.09.2026	tabel FunktsiooniUurimised valemi ja Teema 8 vastuste väljadega
UurimiseKysimused	07.10.2026	uued väljad: muutumispiirkond, asümptoodid ja Teema 11 vastused
FunktsiooniLiik	07.10.2026	väli Liik ja indeks liigi järgi filtreerimiseks

Et rakendus käivituks ühe käsuga, rakendatakse ootel migratsioonid käivitamisel automaatselt (joonis 4). Kui andmebaas on tühi, lisatakse näidisfunktsioonid ja kursuse ülesannete funktsioonid. Kirjed, mis salvestati enne uute küsimuste lisamist, analüüsitakse uuesti, et ka neil oleksid kõik vastused olemas.

```
using (var ulatus = app.Services.CreateScope())
{
    var kontekst =
ulatus.ServiceProvider.GetRequiredService<FunktsioonideKontekst>();
    var analyysija =
ulatus.ServiceProvider.GetRequiredService<FunktsiooniAnalyysija>();

    await kontekst.Database.MigrateAsync();
    await Naidisandmed.LisaAsync(kontekst, analyysija);
    await Naidisandmed.UuendaVanadAsync(kontekst, analyysija);
}
```

Joonis 4. Migratsioonide rakendamine ja näidisandmete lisamine käivitamisel (Program.cs)

2.4.3. CRUD-loogika

CRUD-toimingud on realiseeritud kontrolleris `FunktsioonidController`. Nimekirja vaates loetakse kõigepealt iga liigi funktsioonide arv filtrinuppude jaoks, seejärel funktsioonid ise, vajaduse korral ühe liigi kaupa filtreerituna (joonis 5). Päringud koostatakse LINQ-iga ning EF Core teisendab need SQL-iks; filtreerimine ja rühmitamine tehakse andmebaasis.

```
var loendus = await _kontekst.FunktsiooniUurimised
    .GroupBy(f => f.Liik)
    .Select(g => new { Liik = g.Key, Arv = g.Count() })
    .ToDictionaryAsync(g => g.Liik, g => g.Arv);

var paring = _kontekst.FunktsiooniUurimised.AsQueryable();
if (!string.IsNullOrEmpty(liik))
    paring = paring.Where(f => f.Liik == liik);

var funktsioonid = await paring.OrderByDescending(f =>
f.LuodudAeg).ToListAsync();
```

Joonis 5. Funktsioonide lugemine ja liigi järgi filtreerimine (FunktsioonidController.Index)

Uue funktsiooni loomisel valideeritakse vorm, analüüsitakse valem ja kantakse vastused olemisse. Seejärel lisatakse olem kontekstile ja salvestatakse muudatused (joonis 6). Kui valem on vigane, lisatakse analüüsija veateade vormi olekusse ja kasutajale näidatakse vormi uuesti koos veateatega. Muutmine toimib samamoodi, kuid olemasolev olem leitakse meetodiga `FindAsync` ja selle väljad kirjutatakse üle.

```
[HttpPost, ValidateAntiForgeryToken]
public async Task<IActionResult> Create(FunktsiooniVorm vorm)
{
    var tulemus = Analyysi(vorm);
    if (tulemus is null)
        return View(vorm);

    var olem = new FunktsiooniUurimine();
    tulemus.KannaOlemisse(olem);

    await _kontekst.FunktsiooniUurimised.AddAsync(olem);
    await _kontekst.SaveChangesAsync();

    TempData["Teade"] = "Funktsioon on analüüsitud ja salvestatud.";
    return RedirectToAction(nameof(Details), new { id = olem.Id });
}
```

Joonis 6. Funktsiooni analüüsimine ja salvestamine (FunktsioonidController.Create)

Kustutamine on kahesammuline: GET-päring näitab kinnituslehte ja alles POST-päring kustutab kirje (joonis 7). Nii ei saa kirjet kustutada lingi kaudu ning kõik andmeid muutvad päringud on kaitstud võltsimisvastase märgisega (`ValidateAntiForgeryToken`). Pärast

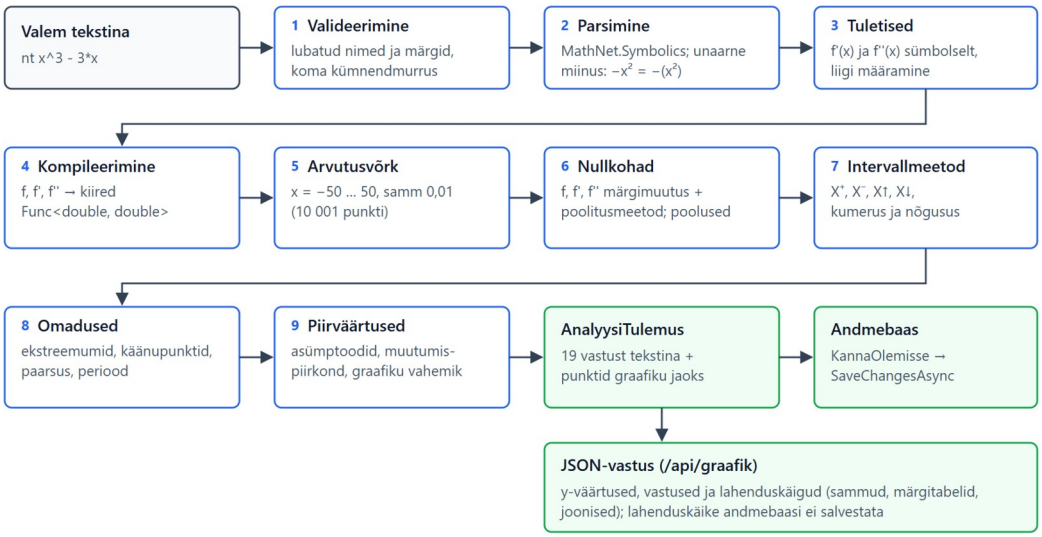
salvestamist suunatakse kasutaja ümber (*Post/Redirect/Get*), et lehe värskendamine ei saadaks vormi uuesti.

```
[HttpPost, ActionName("Delete"), ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var olem = await _kontekst.FunktsiooniUurimised.FindAsync(id);
    if (olem is not null)
    {
        _kontekst.FunktsiooniUurimised.Remove(olem);
        await _kontekst.SaveChangesAsync();
        TempData["Teade"] = "Funktsioon on kustutatud.";
    }
    return RedirectToAction(nameof(Index));
}
```

Joonis 7. Funktsiooni kustutamine (FunktsioonidController.DeleteConfirmed)

2.4.4. Funktsiooni analüüs

Funktsiooni analüüs on rakenduse keskne osa. Selle etapid on kujutatud joonisel 8.



Joonis 8. Funktsiooni analüüsi etapid

Kõigepealt kontrollitakse valemit: lubatud on ainult muutuja *x*, funktsioonid `sin`, `cos`, `tan`, `exp`, `ln` ja `sqrt` ning konstandid `pi` ja `e`. Koma kümnendmurrus ja tundmatud nimed annavad eraldi veateate. Seejärel parsitakse valem teegiga MathNet.Symbolics. Arenduse käigus ilmnes, et teek tõlgendab avaldist `-x^2` kujul $(-x)^2$, mis on matemaatiliselt vale: koolis tähendab $-x^2$ avaldist $-(x^2)$. Probleem lahendati regulaaravaldisega, mis asendab unaarse miinuse enne parsimist korrutisega `(-1)*` (joonis 9).

```
// Unaarne miinus valemi alguses, sulu või tehtemärgi järel – MathNet loeks
-x^2 kui (-x)^2 = x^2.
[GeneratedRegex(@"(?<=^\s*|[(+\-*]\s*)-")]
private static partial Regex UnaarneMiinus();

avaldis = SymbolicExpression.Parse(UnaarneMiinus().Replace(valem, "(-1)*"));
```

Joonis 9. Unaarse miinuse asendamine enne parsimist (FunktsiooniAnalyysija.cs)

Pärast parsimist leitakse esimene ja teine tuletis sümbolselt ning kõik kolm avaldist kompileeritakse funktsioonideks. Need arvutatakse vahemikus [−50; 50] sammuga 0,01, s.t 10 001 punktis. Selle arvutusvõrgu põhjal leitakse määramispiirkond, poolused, funktsiooni ja tuletiste nullkohad ning märgipiirkonnad. Ekstreemumid leitakse kriitilistes punktides tuletise märgimuutuse järgi: kui tuletis on kriitilisest punktist vasakul positiivne ja paremal negatiivne, on seal maksimum, vastupidisel juhul miinimum. Käänupunktid leitakse samamoodi teise tuletise märgimuutuse järgi.

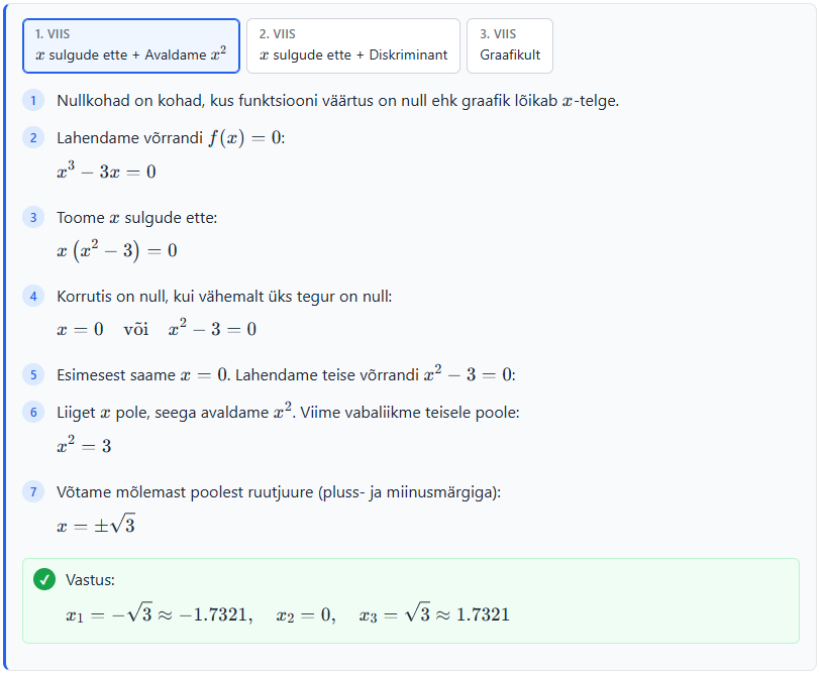
Analüüsi tulemus on objekt `AnalyysiTulemus`, mis sisaldab kõiki vastuseid tekstina ning graafiku jaoks punktide koordinaate ja soovitatud x-vahemikku. Soovitatud vahemik valitakse nii, et nullkohad, ekstreemumid, käänupunktid ja asümptoodid oleksid graafikul näha; perioodilise funktsiooni korral kuvatakse vähemalt kaks perioodi.

2.4.5. Lahenduskäigud

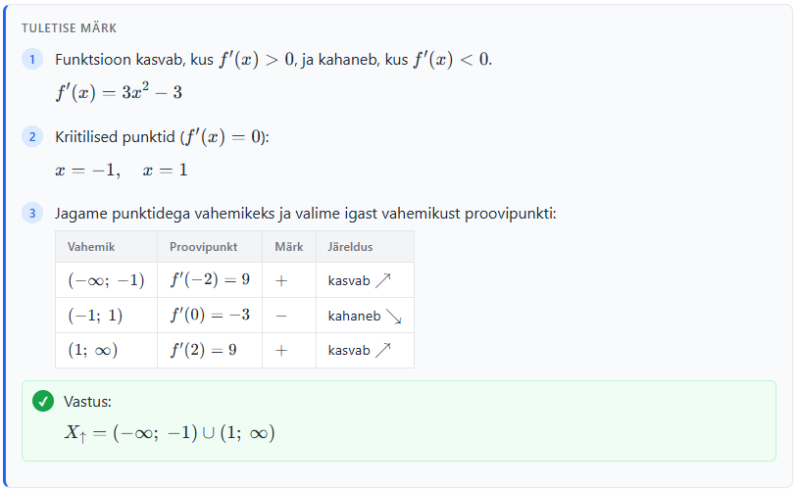
Lahenduskäigud koostab klass `FunktsiooniAnalyysija.Lahendused`. Iga küsimuse kohta koostatakse üks või mitu lahendusviisi, millest igaüks koosneb sammudest. Samm võib sisaldada teksti, eraldi real olevat valemit, märgitabelit või joonist. Valemid koostatakse LaTeX-kujul ning brauseris kuvab need KaTeX. Võrrandite lahendamiseks on realiseeritud järgmised koolivõtted:

- lineaarvõrrand, ruutvõrrand (diskriminant, taandatud ruutvõrrandi valem, Viète'i teoreem) ja mittetäielik ruutvõrrand;
- x sulgude ette toomine, asendus $t = x^2$ (biruutvõrrand) ja Horneri skeem kõrgema astme polünoomi korral;
- murd (lugeja võrdub nulliga) ja korrutis (vähemalt üks tegur võrdub nulliga);
- logaritmi ja eksponendi omadused ning trigonomeetriliste võrrandite üldlahend;
- numbriline lahendamine, kui ükski koolivõte ei sobi.

Joonisel 10 on näha funktsiooni $x^3 - 3x$ nullkohtade leidmine kolmel viisil. Lahendid on esitatud täpsel kujul ($\pm\sqrt{3}$). Märgipiirkondade, monotoonsuse ja kumeruse lahenduskäigus koostatakse märgitabel, milles on iga vahemiku proovipunkt, selles arvutatud väärtus ja järeldus (joonis 11).



Joonis 10. Nullkohtade lahenduskäik mitme lahendusviisiga



Joonis 11. Kasvamisvahemike lahenduskäik märgitabeliga

Tuletise lahenduskäik kirjutatakse lahti reeglite kaupa (summa, astme, korrutise, jagatise ja liitfunktsiooni reegel) ning tulemust kontrollitakse teegi leitud tuletisega. Paarsuse lahenduskäigus asendatakse x avaldisega $-x$ ja võrreldakse tulemust esialgse funktsiooniga; asümptootide korral arvutatakse piirväärtused. Viis „Graafikult“ näitab väikest graafikut,

millel on märgitud nullkohad; positiivsus- ja negatiivsuspiirkonna korral on värvitud ala, kus $f(x) > 0$ või $f(x) < 0$ (joonis 12).



Joonis 12. Nullkohtade lugemine graafikult lahenduskäigu joonisel

2.4.6. Graafik ja eelvaade

Graafiku joonistab skript `graafik.js` teegiga Chart.js. Brauser genereerib valitud vahemikus x-väärtused ja saadab need koos valemiga serverisse (`POST /api/graafik`). Server tagastab y-väärtused, oluliste punktide koordinaadid, asümptoodid, vastused ja lahenduskäigud (vt lisa 2). Valmis graafik on esitatud joonisel 13.



Joonis 13. Funktsiooni x³ − 3x detailvaade ja graafik

Pärast suumimist või liigutamist arvutab server punktid uue vahemiku jaoks (umbes 800 punkti), nii et joon on sisse suumides terav ning ulatub välja suumides üle kogu graafiku. Vertikaalse asümptoodi eri pooltel olevaid punkte ei ühendata, et graafikule ei tekiks vale

püstjoont. *y*-telje ulatus valitakse oluliste punktide järgi, et näiteks funktsiooni $x^4 - 4x^3$ miinimum ei jääks servades kiiresti kasvavate väärtuste kõrval nähtamatuks.

Loomise ja muutmise vormis uuenevad graafik ja vastused valemi sisestamise ajal. Et iga klahvivajutus ei tekitaks serverile päringut, tehakse päring alles siis, kui kasutaja on 350 millisekundit kirjutamata olnud. Kui vahepeal alustati uut päringut, jäetakse vanema päringu vastus kasutamata.

Visuaalne valemiväli (MathLive) ja tekstiväli on sünkroonis: valemi muutmisel ühes väljas teisendatakse see teise välja kujule (näiteks `\frac{x+2}{x-1}` ↔ `(x + 2)/(x - 1)`). Serverisse salvestatakse alati tekstikuju. Kui MathLive ei laadi, jääb alles ainult tekstiväli ning rakendus töötab edasi.

2.4.7. Versioonihaldus

Lähtekoodi muudatusi hallati versioonihaldussüsteemiga Git. Hoidlast on välja jäetud ehitamise vahefailid, arenduskeskkonna failid ja andmebaasifail (`.gitignore`), sest andmebaas luuakse käivitamisel migratsioonide põhjal uuesti. Rakenduse lähtekood on kättesaadav aadressil [lisada koodihoidla link].

2.5. Valminud lahenduse testimine

Rakendust testiti funktsionaalselt: kontrolliti, kas rakendus annab etteantud funktsioonide korral õiged vastused ja kas vigane sisend annab arusaadava veateate. Testimiseks kasutati kursuse ülesannete funktsioone ning funktsioone, mis esindavad kõiki funktsiooni liike ja erijuhte (poolus, piiratud määramispiirkond, perioodilisus, koolivõtetega lahendamatu võrrand). Vastuseid võrreldi käsitsi leitud vastustega. Testid tehti rakendusliidese `POST /api/graafik` kaudu ning kasutajaliideses. Tulemused on esitatud tabelis 10.

Tabel 10. Funktsionaalse testimise tulemused

Valem	Kontrollitud vastused	Tulemus
x^3 - 3*x	kuupfunktsioon; X_0: ±1,7321 ja 0; paaritu; max (−1; 2), min (1; −2); käänupunkt (0; 0); $X\uparrow = (-\infty; -1) \cup (1; \infty)$	õige
x^2 - 4*x + 3	ruutfunktsioon; X_0: 1 ja 3; Y = [−1; ∞); min (2; −1); 4 lahendusviisi (diskriminant, taandatud valem, Viète, graafik)	õige
1/(x - 1)	$X = \mathbb{R} \setminus \{1\}$; $Y = \mathbb{R} \setminus \{0\}$; asümptoodid $x = 1$ ja $y = 0$; nullkohti ja ekstreemumeid pole	õige
sqrt(x)	juurfunktsioon; X = [0; ∞); Y = [0; ∞); ei ole paaris ega paaritu (X pole sümmeetriline)	õige

Valem	Kontrollitud vastused	Tulemus
x*exp(-x)	X_0: 0; max (1; 0,3679); käänupunkt (2; 0,2707); horisontaalne asümptoot $y = 0$	õige
(x^2 - 1)/(x^2 + 1)	paarisfunktsioon; Y = [−1; 1); min (0; −1); käänupunktid (±0,5774; −0,5); asümptoot $y = 1$	õige
sin(x)	perioodiline, T = 2π; Y = [−1; 1]; paaritu; nullkohtade üldlahend	õige
x^3 + x + 1	X_0: −0,6823 (koolivõtteid pole, näidatakse numbrilist lahendamist); ekstreemumeid pole	õige

Teise rühma testidega kontrolliti valemi valideerimist ja veateateid (tabel 11). Kõigil juhtudel anti eestikeelne veateade, mis selgitab, kuidas viga parandada (joonis 17).

Tabel 11. Valemi valideerimise testid

Sisend	Oodatud käitumine	Saadud veateade
3x	korrutusmärgi puudumine	„Valemit ei õnnestunud lugeda. Kontrolli sulge ja tehtemärke (nt 3*x, mitte 3x).“
x,5	koma kümnendmurrus	„Kasuta kümnendkohtade jaoks punkti, mitte koma (nt 0.5*x).“
y+1	tundmatu muutuja	„Tundmatu nimi "y". Muutuja peab olema x; …“
sin(x	sulgemata sulg	„Valemit ei õnnestunud lugeda. …“
ln(x-100)	funktsioon pole uuritavas vahemikus määratud	„Funktsioon ei ole vahemikus [-50; 50] üheski punktis määratud.“

CRUD-toimingute kontrollimiseks loodi vormi kaudu funktsioon `x^2 - 9`, mille detailvaates olid nullkohad −3 ja 3; seejärel muudeti valemiks `x^2 - 16` (nullkohad −4 ja 4) ning kirje kustutati, misjärel selle detailvaade tagastas oleku 404. Ilma võltsimisvastase märgiseta saadetud POST-päring lükati tagasi olekuga 400, mis kinnitab nõude M6 täitmist. Jõudluse kontrollimiseks mõõdeti rakendusliidese vastuse aega kohalikus arvutis: analüüs koos kõigi lahenduskäikudega võttis 18–37 millisekundit, mis täidab mittefunktsionaalse nõude M2.

Testimise käigus ilmnesid rakenduse piirangud. Analüüs tehakse vahemikus [−50; 50], mistõttu väljaspool seda vahemikku asuvaid nullkohti ja ekstreemume ei leita ning lõpmatuses käitumine järeldatakse vahemiku servade põhjal. Ligikaudsed vastused ümardatakse nelja kümnendkohani. Rakendusele ei ole kirjutatud automaatteste; testimine tehti käsitsi ning automaattestide lisamine on üks edasiarenduse võimalusi.

2.6. Kasutusjuhend

2.6.1. Paigaldamine ja käivitamine

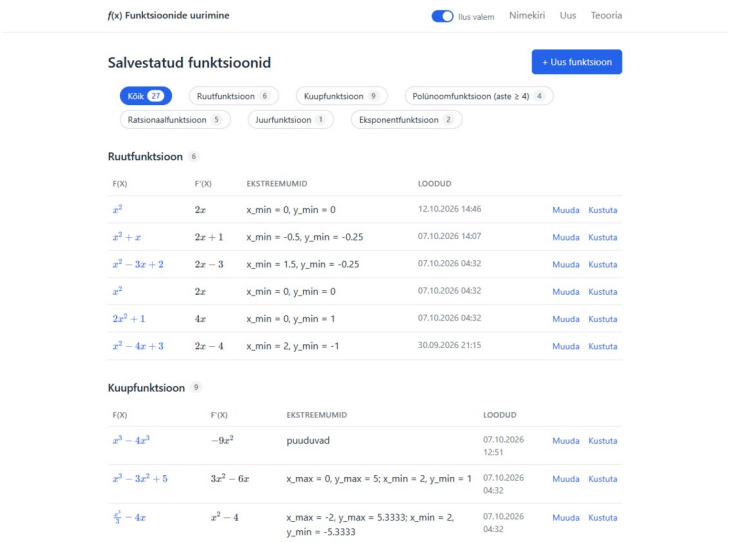
Rakenduse käivitamiseks on vaja .NET 10 SDK-d. Eraldi andmebaasiserverit ega muid programme ei ole vaja. Rakendus paigaldatakse ja käivitatakse järgmiselt:

1. Paigaldada .NET 10 SDK (https://dotnet.microsoft.com/download).
2. Laadida alla rakenduse lähtekood ja avada käsurida projekti kaustas.
3. Käivitada rakendus käsuga `dotnet run`.
4. Avada brauseris aadress http://localhost:5080.

Esmakäivitusel luuakse andmebaasifail `funktsioonid.db`, rakendatakse migratsioonid ja lisatakse näidisfunktsioonid. Migratsioone saab hallata ka käsitsi (joonis 3). Graafiku, valemite ja valemivälja kuvamiseks peab arvuti olema ühendatud internetiga, sest JavaScripti teegid laaditakse CDN-ist.

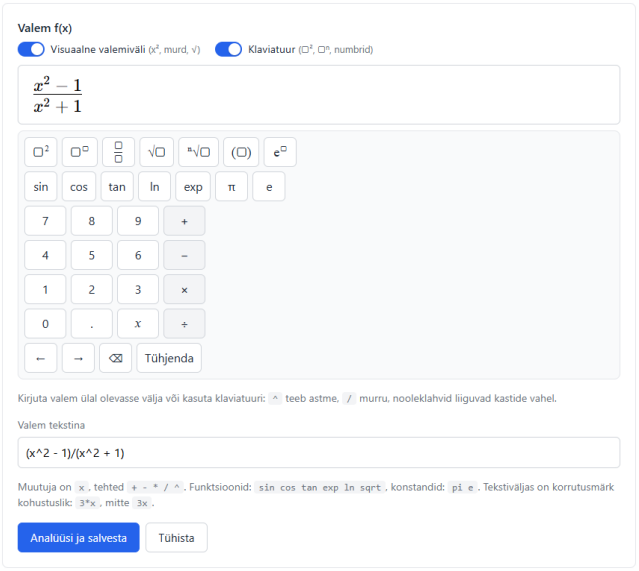
2.6.2. Rakenduse kasutamine

Avalehel on salvestatud funktsioonide nimekiri, mis on rühmitatud liikide kaupa (joonis 14). Ülal olevate nuppudega saab näidata kõiki funktsioone või ainult ühte liiki. Funktsiooni valemil klõpsates avaneb detailvaade; linkidega „Muuda“ ja „Kustuta“ saab funktsiooni muuta või kustutada.



Joonis 14. Salvestatud funktsioonide nimekiri

Uue funktsiooni lisamiseks tuleb vajutada nuppu „+ Uus funktsioon“ ja sisestada valem (joonis 15). Valemi saab kirjutada visuaalsesse valemivälja (klahv `^` teeb astme, `/` murru) või sisestada valemi klaviatuuri nuppudega. Valemi võib kirjutada ka tekstiväljale, kus korrutusmärk on kohustuslik (`3*x`, mitte `3x`). Lülititega saab visuaalse valemivälja ja klaviatuuri peita. Sisestamise ajal uuenevad allpool eelvaate graafik ja vastused. Nupp „Analüüsi ja salvesta“ salvestab funktsiooni ning avab selle detailvaate.



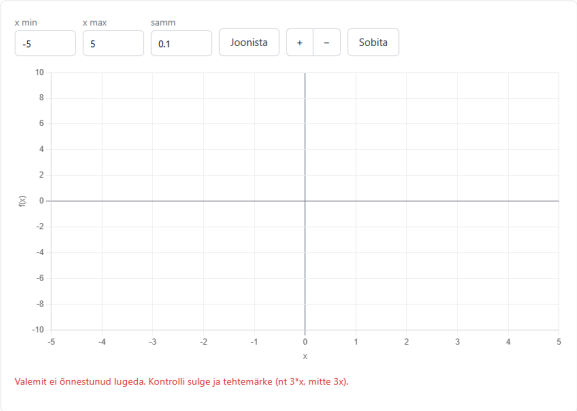
Joonis 15. Uue funktsiooni sisestamine valemi klaviatuuriga

Detailvaates on graafik ja vastused (joonis 16). Graafikut saab suumida kerimisrattaga või nuppudega „+“ ja „−“ ning liigutada hiirega lohistades. Shift-klahvi all hoides ja lohistades saab suumida valitud alale. Nupp „Sobita“ või topeltklõps viib graafiku tagasi algvaatele. Legendil klõpsates saab punktiliike peita. Vastuse juures oleva nupuga „Lahendus“ avaneb lahenduskäik; mitme lahendusviisi korral saab viisi valida vahelehtedel.



Joonis 16. Funktsiooni uurimise vastused koos lahenduskäigu nuppudega

Kui valem on vigane, näidatakse graafiku all veateadet, mis selgitab, kuidas viga parandada (joonis 17). Päises olev lüliti „Ilus valem“ määrab, kas valemeid näidatakse matemaatilisel kujul ($x^3 - 3x$) või kirjutatud kujul (`x^3 - 3*x`). Valik jäetakse brauserisse meelde.



Joonis 17. Veateade vigase valemi korral

Menüüpunkt „Teooria“ avab kursuse teemade 1–12 kokkuvõtte valemitega. Näidete juures olevad nupud „Proovi“ avavad uue funktsiooni vormi, kuhu on näite valem juba sisestatud.

KOKKUVÕTE

Töö eesmärk oli luua veebirakendus, mis sisestatud valemi põhjal uurib funktsiooni, joonistab selle graafiku, selgitab iga vastuse lahenduskäiku ning salvestab uuritud funktsioonid andmebaasi. Eesmärk saavutati: valminud rakendus vastab funktsiooni uurimise 19 küsimusele, määrab funktsiooni liigi, joonistab graafiku koos oluliste punktide ja asümptootidega ning näitab iga vastuse juures lahenduskäiku. Salvestatud funktsioone saab vaadata, muuta ja kustutada ning nimekirjas liigi järgi filtreerida.

Töö esimeses peatükis koostati lähteülesanne, valiti tehnoloogiad ning kavandati rakenduse arhitektuur, andmemudel ja kasutajaliides. Valitud tehnoloogiad osutusid sobivaks: ASP.NET Core MVC võimaldas eraldada kasutajaliidese, äriloogika ja andmetele juurdepääsu, EF Core koos SQLite'iga tegi andmebaasi loomise ja muutmise migratsioonidega lihtsaks ning MathNet.Symbolics võimaldas valemeid sümbolselt töödelda.

Teises peatükis kirjeldati arendusprotsessi ja testimist. Töö käigus jõuti järgmiste järeldusteni:

- sümbolarvutus ja numbriline analüüs täiendavad teineteist: numbriline analüüs leiab vastused ka siis, kui võrrandit koolivõtetega lahendada ei saa, sümbolarvutus annab täpse kuju ja lahenduskäigu;
- täpse vastuse kontrollimine numbrilise vastu välistab vale täpse vastuse näitamise;
- valmisteekide käitumist tuleb kontrollida valdkonna reeglite järgi – teegi tõlgendus avaldisele `-x^2` erines koolimatemaatika reeglist ja vajas parandamist;
- eelvaade, mis uueneb valemi sisestamise ajal, eeldab kiiret analüüsi; mõõdetud vastuseaeg jäi alla 40 millisekundi.

Testimine näitas, et rakendus annab kontrollitud funktsioonide korral õiged vastused ja arusaadavad veateated. Rakenduse piirang on, et analüüs tehakse vahemikus $[-50; 50]$. Edasiarenduseks on autori hinnangul järgmised võimalused:

- automaattestide lisamine analüüsijale ja lahenduskäikudele, et vältida muudatuste käigus tekkivaid vigu;
- puutuja ja normaali ning lõigu suurima ja vähima väärtuse leidmise lisamine kasutajaliidesesse (rakendusliides on juba olemas);

– graafiku joonise lisamine ka teiste vastuste (ekstreemumid, monotoonsus, kumerus) lahenduskäikudesse;
– kasutajakontod, et iga õppija näeks ainult oma funktsioone;
– JavaScripti teekide serveerimine rakendusest endast, et rakendus töötaks täielikult ka ilma internetiühenduseta.

KASUTATUD ALLIKAD

Anthropic. (2026). *Claude* [Suur keelemudel]. https://claude.ai

Chart.js. (s.a.-a). *Chart.js documentation*. https://www.chartjs.org/docs/latest/

Chart.js. (s.a.-b). *chartjs-plugin-zoom*. https://www.chartjs.org/chartjs-plugin-zoom/latest/

KaTeX. (s.a.). *KaTeX documentation*. https://katex.org/docs/

Math.NET. (s.a.). *Math.NET Symbolics* [Lähtekoodihoidla]. GitHub. https://github.com/mathnet/mathnet-symbolics

MathLive. (s.a.). *MathLive*. https://mathlive.io/

Microsoft. (2025, 7. november). *What's new in .NET 10*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview

Microsoft. (s.a.-a). *Dependency injection in ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection

Microsoft. (s.a.-b). *Migrations overview*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/

Microsoft. (s.a.-c). *Overview of ASP.NET Core MVC*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

Microsoft. (s.a.-d). *Overview of Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/

Microsoft. (s.a.-e). *Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/scrum-guide.html

SQLite. (s.a.). *About SQLite*. https://www.sqlite.org/about.html

LISAD

Lisa 1. Uurimisküsimused ja näidisvastused

Lisa 2. Graafiku rakendusliidese päringu ja vastuse näide

Lisa 1. Uurimisküsimused ja näidisvastused

Tabelis 12 on kõik küsimused, millele rakendus vastab, koos funktsiooni $f(x) = x^3 - 3x$ vastustega. Vastused on rakenduse väljund.

Tabel 12. Uurimisküsimused ja vastused funktsiooni $x^3 - 3x$ korral

Rühm	**Küsimus**	**Vastus**
Uurimine	Funktsiooni liik	Kuupfunktsioon
Uurimine	Määramispiirkond X	$(-\infty; \infty)$
Uurimine	Muutumispiirkond Y	$(-\infty; \infty)$
Uurimine	Nullkohad X_0	x1 = −1,7321, x2 = 0, x3 = 1,7321
Uurimine	Positiivsuspiirkond X^+	$(-1{,}7321; 0) \cup (1{,}7321; \infty)$
Uurimine	Negatiivsuspiirkond X^-	$(-\infty; -1{,}7321) \cup (0; 1{,}7321)$
Uurimine	Paarsus	paaritu funktsioon: f(−x) = −f(x)
Uurimine	Perioodilisus	ei ole perioodiline
Uurimine	Lõikepunkt y-teljega	(0; 0)
Uurimine	Asümptoodid	puuduvad
Tuletise abil	Tuletis f′(x)	$3x^2 - 3$
Tuletise abil	Kriitilised punktid	x1 = −1, x2 = 1
Tuletise abil	Kasvamisvahemikud X↑	$(-\infty; -1) \cup (1; \infty)$
Tuletise abil	Kahanemisvahemikud X↓	(−1; 1)
Tuletise abil	Ekstreemumkohad X_e	x_max = −1, y_max = 2; x_min = 1, y_min = −2
Tuletise abil	Teine tuletis f″(x)	6x
Tuletise abil	Kumer ∩ (f″(x) < 0)	$(-\infty; 0)$
Tuletise abil	Nõgus ∪ (f″(x) > 0)	$(0; \infty)$
Tuletise abil	Käänupunktid	(0; 0)

Lisa 2. Graafiku rakendusliidese päringu ja vastuse näide

Päring `POST /api/graafik` funktsiooni $x^2 - 4x + 3$ kohta nelja x-väärtusega. Vastusest on näidatud valik välju; lahenduskäikudest on toodud ainult nullkohtade esimese lahendusviisi kolm esimest sammu.

Päring:

```
{ "valem": "x^2 - 4*x + 3", "x": [0, 1, 2, 3] }
```

Vastus (lühendatud):


```
{
  "valem": "x^2 - 4*x + 3",
  "y": [3, 0, -1, 0],
  "nullkohad": [1, 3],
  "ekstreemumid": [{ "x": 2, "y": -1, "tyyp": "min" }],
  "kaanupunktid": [],
  "asumptoodid": [],
  "yLoikepunkt": 3,
  "soovitatudVahemik": { "xMin": -1, "xMax": 5 },
  "vastused": {
    "liik": "Ruutfunktsioon",
    "nullkohad": "x1 = 1, x2 = 3",
    "tuletis": "2*x - 4",
    "ekstreemumid": "x_min = 2, y_min = -1"
  },
  "lahendused": {
    "nullkohad": {
      "viisid": [{
        "nimi": "Diskriminant",
        "sammud": [
          { "tekst": "Nullkohad on kohad, kus funktsiooni väärtus on null ehk
graafik lõikab $x$-telge.", "valem": null },
          { "tekst": "Lahendame võrrandi $f(x) = 0$:", "valem": "x^{2} - 4x +
3 = 0" },
          { "tekst": "Ruutvõrrandi $ax^2 + bx + c = 0$ kordajad: $a = 1$, $b
= -4$, $c = 3$.", "valem": null }
        ]
      }]
    }
  }
}
```